

Course Code: CSE 499A

Section:15

Project Group No: Group 06

Group Members:

Md.Murad Ul Momin (ID: 2232360642)

Rima Saha (ID: 2132517042)

Sakia Tabassum Anonna (ID: 2221433042)

Ashraful Hossain Riyan (ID: 2232845642)

PRIVATE SHIELD: A Privacy-Preserving Federated Intelligence System for IoT Threat Detection

Final Year Project & Comprehensive System Proposal

ABSTRACT

Smart homes and consumer IoT environments are growing exponentially. Yet, billions of edge devices remain structurally insecure and highly vulnerable to hijacking by botnets due to limited processing budgets and the absence of hardware filters. Legacy Centralized Intrusion Detection Systems (CIDS) mitigate these threats by continuously exfiltrating raw network telemetry blocks to a centralized server, thereby exposing underlying systems' networks to severe private data disclosures and processing latency. This project presents PRIVATE SHIELD, an adversarial-robust, privacy-preserving federated intelligence optimization loop designed exclusively for edge-layer threat classification. By establishing a decentralized Federated Learning engine powered by the FedAvg averaging algorithm, heterogeneous nodes collectively derive global parameters without externalizing sensitive unencrypted packets. To guarantee cryptographic anonymity, formal Differential Privacy (DP) perturbations are injected directly into localized gradients. Uniquely, this framework introduces an active adversarial research cycle that deliberately evaluates system convergence thresholds against data poisoning, free-rider node exploitation, and inference extraction attacks to build unified structural defenses. The final architectural package integrates a live React Security Operations Center (SOC) visualization layer backed by WebSockets and an automated Large Language Model (LLM) assistant to synthesize system events into forensic remediation logs.

I. INTRODUCTION

The contemporary processing landscape is undergoing an unprecedented transformation driven by the integration of the Internet of Things (IoT) into consumer, medical, and industrial automation environments. Pervasive environments now integrate an array of edge apparatuses, including surveillance cameras, household sensors, medical equipment, and smart appliances. Despite this widespread integration, consumer and industrial IoT nodes are

predominantly designed with minimal internal security, limited computing budgets, and the absence of hardware security modules. As a result, these nodes are completely vulnerable to coordinated external exploitation and runtime compromises.

Historically, identifying threat signatures across distributed node topologies required continuously transmitting unencrypted transactional data lakes to a centralized remote cloud database cluster. However, this centralized exfiltration paradigm creates immediate points of failure in security. Continuous streaming of sensitive telemetry introduces critical user data disclosures, demands unsustainable network bandwidth allocations, and exposes the overall system to complete communication blackouts during wide-area network link drops. To establish an optimized alternative, this document presents the structural design criteria for PRIVATE SHIELD, a localized, distributed threat tracking intelligence framework.

II. PROBLEM STATEMENT

Traditional Intrusion Detection Systems (IDS) are fundamentally built upon centralized architectural paradigms that are no longer viable under modern security, privacy, and infrastructure constraints. In these legacy setups, resource-constrained Internet of Things (IoT) edge devices are forced to continuously stream raw, high-frequency network packets, system logs, and multi-modal sensor logs across wide-area networks to a centralized cloud-hosted repository. This baseline operational mechanism introduces a massive and systematic vulnerability: the continuous exfiltration of unencrypted telemetry directly exposes intimate behavioral routines, private residential audio/video feeds, industrial operational

details, and confidential enterprise transactions to third-party interception, data-lake breaches, or insider threats.

Simultaneously, the sheer volume of continuous data transmission from billions of interconnected edge nodes strains the underlying network infrastructure. Volumetric telemetry streams cause unsustainable bandwidth exhaustion, severe spikes in network latency, packet degradation, and crippling dependencies on constant, uninterrupted cloud connectivity. When wide-area network links drop, edge networks are left completely blind and exposed, unable to process or flag localized malicious traffic.

Compounding these architectural flaws is the vulnerability of edge intelligence to state-of-the-art Adversarial Machine Learning (AML) vectors. Current security solutions rely on static, isolated models that fail to adapt dynamically to highly heterogeneous, Non-IID (Independent and Identically Distributed) consumer hardware profiles. This lack of cooperative intelligence renders edge networks highly susceptible to coordinated zero-day exploits and multi-vector evasion attacks.

Furthermore, when distributed optimization loops such as Federated Learning are deployed in practice to keep data localized, they introduce new attack surfaces. Malicious or compromised edge nodes can seamlessly perform Data Poisoning attacks by injecting corrupted model parameters into the global pipeline, causing systemic classification decay. Rogue actors can similarly execute Free-Rider exploits to drain global model intelligence without contributing local compute, or leverage Gradient Inference vectors to reverse-engineer and reconstruct private local data directly from intercepted mathematical updates.

Currently, the academic and industrial computing spaces lack an integrated, production-grade security architecture capable of resolving this tension between data privacy and threat visibility. There is a lack of decentralized verification frameworks that can simultaneously enforce formal cryptographic anonymity boundaries, filter out adversarial parameter mutations, and translate low-level cryptographic logs into plain-English diagnostics natively at the edge layer without exceeding the rigid processing, memory, and energy budgets of consumer microcontroller architectures.

III. OBJECTIVES

- **Objective 1:** Establish a synchronized, low-latency Federated Learning topological mesh over physical Raspberry Pi and ESP32 nodes using localized Flower abstraction runtimes.
- **Objective 2:** Inject formal localized Laplace/Gaussian perturbation noise into parameter weight updates via Opacus backends to enforce mathematical confidentiality guarantees.
- **Objective 3:** Execute a robust adversarial evaluation pipeline targeting system resiliency under targeted gradient distortion, data contamination, and model parameter deduction vectors.
- **Objective 4:** Formulate real-time mitigation mechanisms incorporating parameter distance filters and localized peer-to-peer reputation state estimation indices.
- **Objective 5:** Deploy an integrated React Security Operations Center container interacting via zero-latency WebSockets with a Large Language Model forensics analyzer.

IV. SCOPE OF THE PROJECT

Included Components: Localized optimization of compact deep autoencoder models running natively on resource-constrained ARM Cortex and Tensilica microprocessors; decentralized aggregation orchestration leveraging customized FedAvg rules; gradient vector perturbation layers; and automated malicious packet generation over Scapy framework modules.

Excluded Components: Fabrication of bespoke proprietary integrated circuits; deployment of global multi-region cloud load balancers; and maintenance of large-scale commercial storage arrays beyond the physical testbed network structure.

V. PROPOSED METHODOLOGY

To realize publication-grade computer science research depth, the operational system methodology organizes the processing pipeline across three distinct computational loops:

A. Telemetry Extraction & Data Preparation: Localized traffic sniffing is executed via native Scapy hooks embedded on edge gateway interfaces. Raw network packet strings are dynamically processed into structured feature maps within localized CICFlowMeter environments. The resulting vector tracks 80 independent statistical indicators, including sliding-window inter-packet arrival variances, TCP flag distributions, protocol offsets, and directional byte rates.

B. Decentralized Weight Synchronization: Edge microcontrollers run localized Multi-Layer Perceptron Autoencoders built on PyTorch. Rather than exporting raw historical files, the local node tracks optimization layers locally. Calculated parameter gradients are bounded using strict mathematical clipping thresholds before being securely dispatched to a centralized server layer running Flower to execute global model averages via the objective optimization rule:

$$w_{-}(t+1) = \sum_{i=1}^K (n_{-}i / n) * w_{-}(t+1)^i + N(0, \sigma^2)$$

C. Adversarial Simulation Loop: To validate the endurance bounds of the distributed framework, adversarial nodes inject systematic parameter mutations during aggregation cycles. The central aggregation block applies distance-metric validation checks to compute Euclidean spacing across historical profiles, dropping anomalous vectors that deviate from standard variance boundaries.

VI. SYSTEM DESIGN / ARCHITECTURE

The decentralized framework divides functional system dependencies into structured computing layers managed by individual engineering leads:

1) Edge Hardware Orchestration (Person A): Manages the layout of edge development microprocessors, sets up physical I/O sensor pins, establishes local Mosquitto MQTT channels, and compiles cross-compiled firmware strings via the Arduino IDE layer.

2) Decentralized Machine Learning (Person B): Engineers PyTorch core neural models, coordinates local Flower update frequencies, benchmarks model data skew, and optimizes multi-layered autoencoder validation metrics.

3) Adversarial Threat Engineering (Person C): Constructs automated adversarial code modules to trigger malicious gradient updates, mocks data poisoning vectors, manages port sweeps via Nmap, and tracks CVE indicators.

4) Full-Stack Dashboard Orchestration (Person D): Develops the asynchronous FastAPI communications backend, sets up Redis caching pipelines, manages time-series data storage pools via InfluxDB, and hooks the Anthropic Claude API token stream.

VII. TOOLS AND TECHNOLOGIES

- **Computational Hardware:** Raspberry Pi 4 (ARM Cortex-A72 Gateway Node), Tensilica ESP32 NodeMCU architectures, DHT11 sensory modules, PIR motion matrices, and localized Ethernet interface hubs.
- **Optimization Frameworks:** Python Programming Core, PyTorch Core, PyTorch Lightning Modules, Flower (flwr) Abstraction Libraries, Opacus Differential Privacy Engines, Scikit-Learn Classifiers.
- **Backend Infrastructure:** Asynchronous FastAPI, PostgreSQL Database Engine, InfluxDB Time-Series Lakes, Redis High-Speed Cache, Docker Container Registries, Mosquitto MQTT Broker Channels.
- **Visualization Interface:** React.js Web Framework, Tailwind CSS Engines, D3.js Topological Layout Libraries, Recharts Analytics, WebSockets Protocol, Anthropic Claude API Forensics Token Processing.

VIII. EXPECTED OUTCOME

A. Centralized Benchmark Performance Matrix: Comprehensive confusion matrices, memory footprint

charts, and classification latency logs mapping Random Forest, Isolation Forest, and Deep Autoencoders under a centralized tracking state.

B. Hardened Federated Learning Framework: A robust, privacy-hardened distributed source repository containing functional DP noise variables, secure aggregation scripts, and dynamic node update validation loops.

C. Operational SOC Dashboard Portal: A web application featuring live D3 topological connectivity states, zero-latency security alerts via WebSockets, and automated technical forensics summaries compiled into PDF packages.

IX. TIMELINE / WORK PLAN

Week 4: Deploy the physical MQTT routing gateway infrastructure, mount environmental-tracking sensor matrices, and establish low-level communication channels between edge-layer nodes.

Week 5: Configure local packet capture daemons using Scapy hooks and finalize multi-protocol network sniffing scripts to establish live traffic ingest loops.

Week 6: Train and optimize isolated baseline centralized machine learning classifiers (XGBoost, Isolation Forest, Random Forest) on target datasets to establish performance control metrics and submit the formal *Proposed Solution Report*.

Week 7: Initialize the Flower (flwr) decentralized learning engine, orchestrate local training loops on edge gateway nodes, and handle asymmetric local hardware synchronization.

Week 8: Stress-test the federated loop under highly skewed, Non-IID (Independent and Identically Distributed) data profiles to evaluate local model convergence behaviors under non-uniform network distributions and compile a *Progress Report*.

Week 9: Integrate Opacus frameworks to inject localized Laplace and Gaussian differential privacy perturbation noise into gradient vectors to shield model updates from structural inversion risks.

Week 10: Launch simulation tests modeling active parameter contamination, data poisoning, and free-rider exploits while evaluating mathematical Euclidean distance defensive filters to discard malicious client weight matrices.

Week 11: Establish continuous communication links by routing asynchronous WebSockets alerts through a FastAPI backend layer to the React interface, and project demonstration.

Week 12: Audit repository history logs, optimize Redis cache states, run final centralized-versus-decentralized performance cross-validations, and deliver the final presentation.

X. CONCLUSION

The PRIVATE SHIELD architecture offers a highly scalable, mathematically rigorous alternative to traditional centralized intrusion-tracking topologies. Shifting intelligence pipelines directly to local edge devices successfully addresses data leakage vectors while maintaining strong classification rates against modern security exploits. By testing the framework against simulated parameter contamination and malicious extraction profiles, this project delivers significant engineering depth aligned with advanced computer science curricula. The systematic milestone layout guarantees consistent version-controlled output across both semesters, bridging theoretical decentralized security paradigms with resilient, low-power industrial network operations.